Lab Experiment 14

Alpha-Beta Pruning Algorithm for Gaming

Aim

To write a Python program to implement the Alpha-Beta Pruning algorithm for a two-player game.

Objective

The objective of this experiment is to implement the Alpha-Beta Pruning algorithm for game playing. It aims to reduce unnecessary node evaluations and efficiently determine the optimal move.

Algorithm

1. Define the initial game state.
2. Generate all possible moves.
3. Build the game tree up to the terminal states.
4. Assign scores to the terminal states.
5. Initialize Alpha (α) and Beta (β) values.
6. Apply the Minimax algorithm with Alpha-Beta pruning.
7. Prune the remaining branches when $\alpha \geq \beta$.
8. Select the move with the best score for the maximizing player.
9. Display the optimal move and score.

Code

```
# Alpha-Beta Pruning Algorithm for Tic Tac Toe
def winner(board):
    lines = []

    lines.extend(board)

    lines.extend([
        [board[r][c] for r in range(3)]
        for c in range(3)
    ])

    lines.append([board[i][i] for i in range(3)])
    lines.append([board[i][2 - i] for i in range(3)])

    for line in lines:
        if line[0] != ' ' and all(x == line[0] for x in line):
            return line[0]

    if all(board[r][c] != ' '
           for r in range(3)
           for c in range(3)):
        return 'Draw'

    return None
```

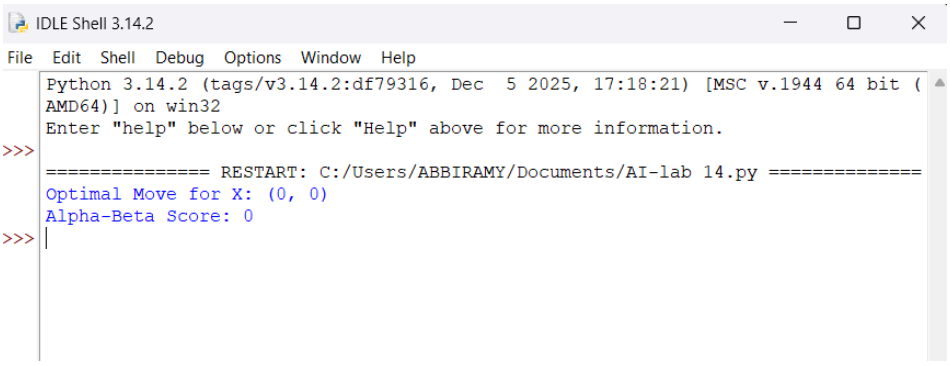
```
def alpha_beta(board, depth, alpha, beta, is_max):

    result = winner(board)

    if result == 'X':
        return 10 - depth

    if result == 'O':
        return depth - 10

    if result == 'Draw':
        return 0

    if is_max:
        best = float('-inf')

        for r in range(3):
            for c in range(3):

                if board[r][c] == ' ':
                    board[r][c] = 'X'

                    value = alpha_beta(
                        board, depth + 1,
                        alpha, beta, False
                    )
```

```
                board[r][c] = ' '

                best = max(best, value)
                alpha = max(alpha, best)

                if alpha >= beta:
                    break

        if alpha >= beta:
            break

    return best

else:
    best = float('inf')

    for r in range(3):
        for c in range(3):

            if board[r][c] == ' ':
                board[r][c] = 'O'

                value = alpha_beta(
                    board, depth + 1,
                    alpha, beta, True
```

```
                )

                board[r][c] = ' '

                best = min(best, value)
                beta = min(beta, best)

                if alpha >= beta:
                    break

            if alpha >= beta:
                break

        return best
def best_move(board):

    best_value = float('-inf')
    move = (-1, -1)

    alpha = float('-inf')
    beta = float('inf')

    for r in range(3):
        for c in range(3):

            if board[r][c] == ' ':
```

```
            board[r][c] = 'X'

            move_value = alpha_beta(
                board, 0, alpha, beta, False
            )

            board[r][c] = ' '

            if move_value > best_value:
                best_value = move_value
                move = (r, c)

            alpha = max(alpha, best_value)

    return move, best_value
board = [[' ' for _ in range(3)] for _ in range(3)]
move, score = best_move(board)
print("Optimal Move for X:", move)
print("Alpha-Beta Score:", score)
```

Output



Result

The Python program successfully implemented the Alpha-Beta Pruning algorithm and determined the optimal move for the given game state.

Conclusion

Alpha-Beta Pruning improves the efficiency of the Minimax algorithm by eliminating branches that do not affect the final decision. It reduces the number of nodes evaluated while producing the same optimal result.